

Weather App

Guia de Estudos

Arquitetura Hexagonal · Java 21 · Spring Boot · Docker
Evolução: APIs externas, Kafka, SonarQube, Cucumber

Documento gerado a partir do estado real do repositório weather-app.
Público: desenvolvedores que estudam backend com Ports & Adapters.

1. Introdução e objetivo do projeto

O Weather App é um serviço backend em Java 21 com Spring Boot 4.0.5, organizado em Arquitetura Hexagonal (Ports & Adapters). O objetivo pedagógico é separar regras de negócio de detalhes técnicos: HTTP, JPA, PostgreSQL e Docker ficam na borda; o núcleo permanece testável e independente de framework.

Neste momento o projeto já possui domínio Weather, portas de entrada/saída, WeatherService, adaptador JPA e endpoint de saúde /hello. Ainda falta o controller REST de clima — o fluxo de negócio está pronto, mas não exposto via HTTP.

1.1 Stack real (pom.xml e docker-compose)

- Java 21 (LTS) — records, var, pattern matching, virtual threads (opcional)
- Spring Boot 4.0.5 — Web MVC, Data JPA, Security, DevTools, Docker Compose support
- PostgreSQL 15 Alpine — banco relacional no Docker
- Maven (mvnw) — build e empacotamento JAR
- Lombok — reduz boilerplate em entidades JPA
- Docker multi-stage — JDK 21 para build, JRE 21 para runtime

2. Java 21 no projeto

2.1 Records como modelo de domínio

A classe `Weather` no pacote `core.domain` é um record: tipo imutável com construtor, acessores e `equals/hashCode` gerados pelo compilador. Records expressam Value Objects ou entidades simples sem comportamento pesado — ideal para o núcleo hexagonal.

```
public record Weather(  
    String city,  
    Double temperature,  
    String description,  
    LocalDateTime timestamp  
) {}
```

Boas práticas: use records no core quando o objeto é principalmente dados; mantenha validações de negócio em métodos estáticos de fábrica ou em serviços de aplicação.

2.2 Injeção de dependência por construtor

`WeatherService` e `WeatherPersistenceAdapter` recebem dependências apenas pelo construtor. Isso torna dependências explícitas, facilita testes unitários com mocks e é preferível a `@Autowired` em campos (field injection).

```
public WeatherService(WeatherRepositoryPort weatherRepositoryPort) {  
    this.weatherRepositoryPort = weatherRepositoryPort;  
}
```

2.3 Optional e exceções

A porta `WeatherRepositoryPort` retorna `Optional<Weather>` em `findByCity` — evita null e deixa explícito que a cidade pode não existir. O `WeatherService` usa `orElseThrow`; em produção, prefira exceções de domínio (ex.: `CityNotFoundException` no core) mapeadas para HTTP 404 na camada infrastructure.

3. Arquitetura Hexagonal aplicada

3.1 Anéis e pacotes

Núcleo (core): domain + ports — sem Spring, sem JPA.

Aplicação (application): implementa casos de uso (GetWeatherUseCase).

Infraestrutura (infrastructure): controllers, config, adapters.out.persistence.

Regra de dependência: setas de import apontam sempre para dentro. Infrastructure → Application → Core. O Core nunca importa infrastructure.

3.2 Portas

Porta de entrada (driving): GetWeatherUseCase — o mundo externo chama execute(city).

Porta de saída (driven): WeatherRepositoryPort — o core pede persistência sem saber se é JPA, arquivo ou API externa.

3.3 Adaptadores

Adaptador de saída: WeatherPersistenceAdapter implementa WeatherRepositoryPort, traduz Weather (domínio) ↔ WeatherEntity (JPA). Este é o Anti-Corruption Layer: o modelo de banco não contamina o domínio.

Adaptador de entrada (a criar): WeatherController chamará GetWeatherUseCase, nunca WeatherService diretamente — desacopla REST da implementação.

3.4 Fluxo de uma requisição GET /weather?city=X

1. Cliente HTTP → WeatherController (infrastructure.controllers)
2. Controller injeta GetWeatherUseCase → execute(city)
3. WeatherService (application) orquestra a regra
4. WeatherRepositoryPort.findByCity → WeatherPersistenceAdapter
5. WeatherJpaRepository → SQL em tb_weather
6. Mapeamento Entity → record Weather e resposta JSON

Validação atual: imports em core.* usam apenas java.* e com.weather_app.core.* — regra de dependência respeitada.

4. Spring Boot e sintaxe usada

4.1 @SpringBootApplication

WeatherAppApplication dispara o contexto IoC: scan de componentes, auto-configuração de DataSource, JPA, Security e embedded Tomcat na porta 8080.

4.2 Estereótipos Spring

- @RestController + @GetMapping — adaptador HTTP (HelloController)
- @Service — WeatherService registrado como bean
- @Component — WeatherPersistenceAdapter
- @Repository — WeatherJpaRepository (Spring Data)
- @Configuration + @Bean — SecurityConfig define SecurityFilterChain

4.3 Spring Data JPA

WeatherJpaRepository extends JpaRepository<WeatherEntity, Long> e declara Optional<WeatherEntity> findByCity(String city). O Spring gera a query pelo nome do método (query derivation) — convenção findBy + Campo.

```
@Entity
@Table(name = "tb_weather")
public class WeatherEntity { ... }
```

4.4 Spring Security

SecurityConfig desabilita CSRF (comum em APIs stateless) e libera apenas /hello; demais rotas exigem autenticação. Ao criar GET /weather, adicione .requestMatchers("/weather/**").permitAll() em dev ou configure JWT/OAuth2 para produção.

4.5 Lombok

@Getter @Setter @NoArgsConstructor @AllArgsConstructor em WeatherEntity: JPA exige construtor vazio; Lombok evita dezenas de linhas. O domínio Weather não usa Lombok — permanece puro.

5. Docker e ambiente local

5.1 docker-compose.yml

Serviço db: imagem postgres:15-alpine, healthcheck com pg_isready, porta 5432.

Serviço backend: build do dockerfile, depende do db healthy, variáveis SPRING_DATASOURCE_*.

5.2 Dockerfile multi-stage

Estágio 1 (build): eclipse-temurin:21-jdk-alpine, ./mvnw package -DskipTests.

Estágio 2 (runtime): JRE 21, copia app.jar e start.sh. Benefício: imagem final menor, sem Maven nem código-fonte em produção.

5.3 start.sh

Script aguarda nc -z \$DB_HOST 5432 antes de java -jar app.jar — evita falha de conexão quando o backend sobe antes do PostgreSQL estar aceitando conexões.

5.4 Comandos úteis

```
docker compose build --progress=plain
docker compose up
curl http://localhost:8080/hello
```

6. Papel de cada classe do projeto

- Weather (core.domain): Modelo de negócio imutável (record).
- GetWeatherUseCase (core.ports): Contrato de caso de uso — porta de entrada.
- WeatherRepositoryPort (core.ports): Contrato de persistência — porta de saída.
- WeatherService (application): Implementa o caso de uso; orquestra regras.
- WeatherPersistenceAdapter (infrastructure): Implementa repositório; mapeia Entity ↔ Domain.
- WeatherEntity (infrastructure): Modelo JPA da tabela tb_weather.
- WeatherJpaRepository (infrastructure): Acesso Spring Data ao banco.
- HelloController (infrastructure): Health check; fora do fluxo de clima.
- SecurityConfig (infrastructure): Política de segurança HTTP.

7. Próximos passos: CRUD e API externa

7.1 Completar o adaptador de entrada

Crie WeatherController com GET /weather/{city}, POST para cadastro e DELETE se necessário. Injete GetWeatherUseCase (interface), não a classe concreta. DTOs de request/response ficam em infrastructure (ex.: WeatherResponse) — nunca no core.

```
@RestController
@RequestMapping("/weather")
public class WeatherController {
    private final GetWeatherUseCase getWeatherUseCase;
    @GetMapping("/{city}")
    public WeatherResponse get(@PathVariable String city) {
        Weather w = getWeatherUseCase.execute(city);
        return WeatherResponse.from(w);
    }
}
```

7.2 CRUD completo na hexagonal

- CreateWeatherUseCase + save na porta — já existe save em WeatherRepositoryPort
- UpdateWeatherUseCase — update na porta ou save com merge
- DeleteWeatherUseCase — nova operação deleteByCity na porta
- ListWeatherUseCase — findAll na porta

Cada operação = uma porta de entrada no core + serviço em application + método no adaptador JPA.

7.3 Consumir API ou microserviço externo

Para clima em tempo real (OpenWeather, etc.), crie nova porta de saída no core, ex.: WeatherProviderPort com Weather fetchCurrent(String city). Implemente WeatherApiAdapter em infrastructure.adapters.out.http usando RestClient ou WebClient.

WeatherService pode combinar: primeiro consulta API externa, depois persiste via WeatherRepositoryPort (cache). O core só enxerga interfaces — troca de provedor não quebra regras.

Microserviços: mesmo padrão — porta no core, cliente HTTP no adaptador; contratos versionados (OpenAPI), timeouts, circuit breaker (Resilience4j) na infrastructure, não no domain.

7.4 Flyway e perfis

- Flyway: migrations versionadas V1__create_weather.sql
- application-dev.properties vs application-prod.properties

- Spring Boot Actuator: /actuator/health para Kubernetes/Docker

8. Primeiros passos com Apache Kafka

Kafka encaixa na hexagonal como adaptador de saída (publicar eventos) ou entrada (consumir). Exemplo: após salvar clima, publicar WeatherUpdatedEvent.

8.1 Conceitos

- Topic — canal de mensagens (ex.: weather.events)
- Producer — adaptador que envia após caso de uso
- Consumer — outro serviço ou listener na infrastructure
- Partition / offset — paralelismo e ordenação por chave (city)

8.2 Integração Spring

Dependência: spring-kafka. Porta no core: WeatherEventPublisherPort. Adaptador: KafkaWeatherEventAdapter com KafkaTemplate. Nunca importe org.apache.kafka no core.

```
# docker-compose (exemplo)
kafka:
  image: confluentinc/cp-kafka:7.5.0
  zookeeper: ... # ou KRaft sem Zookeeper
```

9. SonarQube — qualidade e segurança

SonarQube analisa bugs, vulnerabilidades, code smells e cobertura. Encaixa no pipeline CI após `mvn test jacoco:report`.

9.1 Primeiros passos

- Subir SonarQube via Docker ou SonarCloud (SaaS)
- Gerar token e configurar `sonar.projectKey` no `pom.xml` (plugin `sonar-maven`)
- Executar: `mvn verify sonar:sonar -Dsonar.host.url=...`

Para hexagonal: configure exclusões de cobertura apenas para DTOs e Application main, não para `core.domain` — meta é alta cobertura no core e application.

10. Cucumber — testes BDD

Cucumber expressa comportamento em Gherkin (Given/When/Then), legível para negócio. Testa a aplicação de fora: HTTP + banco (integração) ou mocks das portas (unitário de serviço).

Funcionalidade: Consultar clima

Cenário: Cidade existente

Dado que existe clima cadastrado para "Curitiba"

Quando consulto o clima de "Curitiba"

Então recebo temperatura e descrição

Step definitions em `src/test/java` chamam `MockMvc` ou `Testcontainers PostgreSQL`. Mantenha steps finos; lógica de assert no core testado separadamente com JUnit 5.

11. Estratégia de testes alinhada à arquitetura

- Core/Application: JUnit 5 + Mockito — mock de WeatherRepositoryPort
- Adapter JPA: @DataJpaTest + Testcontainers
- Controller: @WebMvcTest mockando GetWeatherUseCase
- E2E: @SpringBootTest + RestAssured ou MockMvc

O pom já inclui spring-boot-starter-***-test** — expanda além de contextLoads().

12. Roadmap sugerido (ordem de estudo)

- 1. WeatherController + DTO + liberar rota no SecurityConfig
- 2. CRUD completo + tratamento global de erros (@ControllerAdvice)
- 3. Flyway + dados seed
- 4. Adaptador HTTP para API meteorológica externa
- 5. Testes unitários e integração (JUnit + Testcontainers)
- 6. Cucumber para fluxos críticos
- 7. CI (GitHub Actions): build, test, SonarQube
- 8. Kafka para eventos de domínio
- 9. Observabilidade: logs estruturados, métricas, tracing

13. Maven e ciclo de build

O Maven (mvnw no projeto) gerencia dependências transitivas do Spring Boot parent POM. Fases principais: compile → test → package (JAR executável com spring-boot-maven-plugin). O annotationProcessorPaths registra Lombok no compilador — sem isso, getters gerados falham.

```
./mvnw clean verify
./mvnw spring-boot:run
```

spring-boot-docker-compose (runtime): em dev local pode subir o compose automaticamente; em produção use apenas docker compose na raiz do monorepo.

14. JPA e persistência — o que acontece no adaptador

Hibernate (por baixo do Spring Data) sincroniza `WeatherEntity` com `tb_weather`. `@GeneratedValue(IDENTITY)` delega o `id` ao PostgreSQL `SERIAL/BIGSERIAL`. `@Column(unique=true)` em `city` evita duplicatas — alinhado à regra de negócio.

O adaptador nunca retorna `WeatherEntity` para fora da `infrastructure`: sempre converte para `record Weather`. Se amanhã migrar para MongoDB, cria `MongoWeatherAdapter` implementando a mesma porta — `WeatherService` não muda.

15. Como hexagonal se relaciona com DDD e Clean Architecture

Hexagonal $\approx$ Ports & Adapters (Cockburn). Clean Architecture (Uncle Bob) usa camadas Entities / Use Cases / Interface Adapters / Frameworks — mapeamento direto: core.domain = Entities; application = Use Cases; infrastructure = Frameworks; ports = boundaries. DDD agrega Aggregates e Domain Events; um evento WeatherUpdated pode nascer no core e ser publicado por adaptador Kafka sem poluir o domínio.

16. OpenAPI, validação e boas práticas REST

- springdoc-openapi: documentação automática em /swagger-ui
- jakarta.validation @NotBlank no DTO de entrada — validação na borda
- Padrão de erro RFC 7807 (Problem Details) via @ControllerAdvice
- Idempotência em PUT; POST retorna 201 + Location header

Versionamento: /api/v1/weather evita quebrar clientes ao evoluir contratos.

17. Pipeline CI/CD (visão inicial)

```
# Exemplo GitHub Actions (conceitual)
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-java@v4
```

```
        with: { java-version: '21' }
```

```
      - run: ./mvnw -B verify
```

```
      - run: docker build -t weather-backend ./backend
```

SonarQube entra após verify. Imagem Docker publicada em registry; deploy em Kubernetes com probes apontando para Actuator health.

18. Resumo: encaixe ferramenta × camada hexagonal

Java 21 + records → core.domain | Spring @Service → application | @RestController → infrastructure in | JPA + PostgreSQL → infrastructure out | RestClient/API → nova porta out | KafkaTemplate → porta out eventos | Cucumber/MockMvc → testes atravessam borda | SonarQube → qualidade transversal | Docker → empacota infrastructure runtime.

19. Referências rápidas

- Alistair Cockburn — Hexagonal Architecture (original)
- Spring Boot Reference Documentation 4.x
- PostgreSQL 15 Docs — tipos e índices
- Docker Docs — multi-stage builds, compose depends_on
- Apache Kafka Documentation — producers/consumers
- SonarQube Docs — quality gates
- Cucumber.io — Gherkin reference